

Практическое занятие 3. Алгебраические структуры

Учебные вопросы занятий:

1. Группы
2. Кольца
3. Поля

Задание 1. Проверка свойств группы на множестве целых чисел

Цель:

Понять, что такое группа, и проверить, образует ли множество целых чисел $\mathbb{Z}$ группу относительно операции сложения.

Условие:

Проверьте, является ли $(\mathbb{Z}, +)$ группой. Для этого последовательно проверьте все свойства группы:

- Замкнутость
- Ассоциативность
- Наличие нейтрального элемента
- Наличие обратного элемента для каждого элемента

Пример кода:

```
# Проверяем свойства группы для  $(\mathbb{Z}, +)$ 
# Здесь  $\mathbb{Z}$  — целые числа, но в коде ограничимся некоторым диапазоном для проверки
def check_group_addition_Z():
    # Возьмем диапазон для проверки: от -10 до 10
    elements = list(range(-10, 11))
    # Проверка замкнутости
    for a in elements:
        for b in elements:
            c = a + b
            if c not in elements:
                # Теоретически не может случиться, так что пропустим
                pass

    print("Замкнутость: выполнена (целые числа замкнуты по сложению)")
    # Проверка ассоциативности
    for a in elements:
        for b in elements:
            for c in elements:
                if (a + b) + c != a + (b + c):
                    print("Ассоциативность нарушена для", a, b, c)
                    return
    print("Ассоциативность: выполнена")
    # Нейтральный элемент — 0
    neutral = 0
    for a in elements:
        if a + neutral != a or neutral + a != a:
            print("Нейтральный элемент отсутствует")
            return
    print("Нейтральный элемент: 0")
    # Обратный элемент для каждого a — -a
```

```

for a in elements:
    inverse = -a
    if a + inverse != neutral:
        print(f"Обратный элемент отсутствует для {a}")
        return
print("Обратные элементы: существуют для всех элементов")

print("Вывод: (Z, +) образует группу.")

if __name__ == "__main__":
    check_group_addition_Z()

```

Задание 2. Определение, является ли множество группой по умножению

Цель:

Проверить, является ли множество целых чисел Z группой по операции умножения.

Условие:

Проверьте, удовлетворяет ли $(Z, \cdot)$ свойствам группы.

Пример кода:

```

def check_group_multiplication_Z():
    elements = list(range(-10, 11))
    neutral = 1

    # Замкнутость
    for a in elements:
        for b in elements:
            c = a * b
            # Результат всегда целое число, пропускаем проверку

    print("Замкнутость: выполнена (целые числа замкнуты по умножению)")

    # Ассоциативность
    for a in elements:
        for b in elements:
            for c in elements:
                if (a * b) * c != a * (b * c):
                    print("Ассоциативность нарушена")
                    return
    print("Ассоциативность: выполнена")

    # Нейтральный элемент — 1
    for a in elements:
        if a * neutral != a or neutral * a != a:
            print("Нейтральный элемент отсутствует")
            return
    print("Нейтральный элемент: 1")

    # Проверка обратных элементов (кроме 0)
    for a in elements:
        if a != 0:

```

```

inverse_found = False
# Ищем обратный элемент inverse, такой что a * inverse = 1
# В целых числах обратный элемент существует только для 1 и -1
if a == 1 or a == -1:
    inverse_found = True
if not inverse_found:
    print(f"Обратного элемента нет для {a}")
    print("Значит, (Z, *) не является группой")
    return
print("Обратные элементы: существуют не для всех элементов")
print("Вывод: (Z, *) не является группой.")

if __name__ == "__main__":
    check_group_multiplication_Z()

```

Задание 3. Проверка, является ли множество (Z_n) кольцом

Цель:

Понять структуру кольца на примере вычетов по модулю (n) .

Условие:

Пусть $(n=6)$. Рассмотрите множество $(Z_6 = \{0,1,2,3,4,5\})$ с операциями сложения и умножения по модулю 6.

- Проверьте, образует ли (Z_6) кольцо.
- Укажите, есть ли в (Z_6) делители нуля.

Пример кода:

MOD = 6

```

def add_mod(a, b):
    return (a + b) % MOD

def mul_mod(a, b):
    return (a * b) % MOD

def check_ring_Z6():
    elements = list(range(MOD))

    # Проверяем абелеву группу по сложению
    neutral_add = 0

    # Замкнутость по сложению
    for a in elements:
        for b in elements:
            if add_mod(a, b) not in elements:
                print("Не замкнуто по сложению")
                return
    print("Сложение замкнуто")

    # Ассоциативность по сложению

```

```

for a in elements:
    for b in elements:
        for c in elements:
            left = add_mod(add_mod(a, b), c)
            right = add_mod(a, add_mod(b, c))
            if left != right:
                print("Сложение не ассоциативно")
                return
print("Сложение ассоциативно")

# Нейтральный элемент по сложению
for a in elements:
    if add_mod(a, neutral_add) != a or add_mod(neutral_add, a) != a:
        print("Нет нейтрального элемента по сложению")
        return
print("Нейтральный элемент по сложению:", neutral_add)

# Обратный элемент по сложению
for a in elements:
    found_inverse = False
    for x in elements:
        if add_mod(a, x) == neutral_add:
            found_inverse = True
            break
    if not found_inverse:
        print(f"Нет обратного элемента по сложению для {a}")
        return
print("Обратные элементы по сложению: есть для всех")

# Проверка умножения — ассоциативность и дистрибутивность (частично)
# Ассоциативность по умножению
for a in elements:
    for b in elements:
        for c in elements:
            if mul_mod(mul_mod(a, b), c) != mul_mod(a, mul_mod(b, c)):
                print("Умножение не ассоциативно")
                return
print("Умножение ассоциативно")

# Дистрибутивность слева
for a in elements:
    for b in elements:
        for c in elements:
            left = mul_mod(a, add_mod(b, c))
            right = add_mod(mul_mod(a, b), mul_mod(a, c))
            if left != right:
                print("Дистрибутивность слева не выполняется")
                return
print("Дистрибутивность слева выполнена")

# Дистрибутивность справа
for a in elements:

```

```

    for b in elements:
        for c in elements:
            left = mul_mod(add_mod(b, c), a)
            right = add_mod(mul_mod(b, a), mul_mod(c, a))
            if left != right:
                print("Дистрибутивность справа не выполняется")
            return
print("Дистрибутивность справа выполнена")

# Проверка на делители нуля
zero = 0
zero_divisors = []
for a in elements:
    for b in elements:
        if a != zero and b != zero and mul_mod(a, b) == zero:
            zero_divisors.append((a, b))
if zero_divisors:
    print("Найдены делители нуля:", zero_divisors)
else:
    print("Делителей нуля нет")

if __name__ == "__main__":
    check_ring_Z6()

```

Задание 4. Проверка, является ли (Z_p) полем при простом (p)

Цель:

Понять, почему множество вычетов по модулю простого числа образует поле.

Условие:

Пусть ($p=7$). Рассмотрите множество ($Z_7 = \{0,1,2,3,4,5,6\}$) с операциями сложения и умножения по модулю 7.

- Докажите, что (Z_7) образует поле.
- Найдите обратные элементы по умножению для элементов (кроме 0).

Пример кода:

MOD = 7

```

def add_mod(a, b):
    return (a + b) % MOD

def mul_mod(a, b):
    return (a * b) % MOD

def extended_gcd(a, b):
    """Расширенный алгоритм Евклида для поиска обратного по модулю"""
    if b == 0:
        return a, 1, 0
    gcd, x1, y1 = extended_gcd(b, a % b)
    x = y1
    y = x1 - (a // b) * y1

```

```

return gcd, x, y

def mod_inverse(a, m):
    gcd, x, _ = extended_gcd(a, m)
    if gcd != 1:
        return None # Обратного нет
    else:
        return x % m

def check_field_Z7():
    elements = list(range(MOD))
    neutral_add = 0
    neutral_mul = 1

    # Проверка группы по сложению (абелева группа)
    # Аналогично предыдущему заданию, пропускаем детали для краткости

    # Проверка группы по умножению для ненулевых элементов
    non_zero_elements = [e for e in elements if e != 0]

    for a in non_zero_elements:
        inv = mod_inverse(a, MOD)
        if inv is None:
            print(f"Обратного элемента по умножению нет для {a}")
            return
        # Проверка
        if mul_mod(a, inv) != neutral_mul:
            print(f"Неправильный обратный элемент для {a}")
            return

    print("Для всех ненулевых элементов обратные по умножению найдены.")
    print("Значит, Z7 образует поле.")

    # Выведем обратные элементы
    print("Обратные элементы по умножению в Z7:")
    for a in non_zero_elements:
        inv = mod_inverse(a, MOD)
        print(f"{a} -> {inv}")

if __name__ == "__main__":
    check_field_Z7()

```

Задание 5. Исследование подгруппы в группе перестановок

Цель:

Познакомиться с группами перестановок и понять понятие подгруппы.

Условие:

Рассмотрите группу перестановок (S_3) — все перестановки множества $\{1, 2, 3\}$.

Перечислите все элементы (S_3) и найдите подгруппу, состоящую из циклов длины 3.

Пример кода:

```

import itertools

def compose_permutations(p1, p2):
    """Композиция перестановок p1 и p2.
    p1 и p2 — кортежи, где индекс — исходный элемент, значение — куда он переходит."""
    return tuple(p1[p2[i]-1] for i in range(len(p2)))

def find_S3_and_subgroup():
    elements = [1, 2, 3]
    S3 = list(itertools.permutations(elements))

    print("Элементы группы S3 (перестановки):")
    for p in S3:
        print(p)

    # Найдём 3-циклы — циклы длины 3
    # В S3 это (1 2 3) и (1 3 2)
    cycles_3 = []
    for p in S3:
        # Проверим, является ли p 3-циклом
        # Для 3 элементов циклом будет перестановка без неподвижных точек, порядок 3
        # Проверим порядок перестановки
        order = 1
        perm = p
        while True:
            perm = tuple(perm[i-1] for i in perm)
            order += 1
            if perm == p or order > 6:
                break
        if order == 3:
            cycles_3.append(p)

    print("\nПодгруппа, состоящая из 3-циклов и тождества:")
    print("(1,2,3) - идентичность")
    print("(123) и (132) — 3-циклы")
    print("Вычислим их явные представления:")

    # Идентичность
    identity = (1, 2, 3)
    subgroup = [identity] + cycles_3

    for perm in subgroup:
        print(perm)

if __name__ == "__main__":
    find_S3_and_subgroup()

```

Задания для самостоятельного выполнения

Задание 6. Проверка, является ли множество чётных целых чисел группой по сложению

Условие:

- Рассмотрите множество $(2\mathbb{Z} = \{\dots, -4, -2, 0, 2, 4, \dots\})$ — все чётные целые числа.
- Проверьте, образует ли $(2\mathbb{Z}, +)$ группу.
- Проверьте замкнутость, ассоциативность, наличие нейтрального и обратного элементов.

Рекомендации:

Реализуйте проверку аналогично Заданию 1.

Задание 7. Исследование множества неотрицательных целых чисел по умножению

Условие:

Пусть $(\mathbb{N}_0 = \{0, 1, 2, 3, \dots\})$.

- Является ли $(\mathbb{N}_0, \cdot)$ группой?
- Если нет, укажите, какие из аксиом группы нарушаются.

Рекомендации:

Обратите внимание на наличие обратных элементов и нейтрального элемента.

Задание 8. Проверка, является ли множество вычетов $(\mathbb{Z}_{10})$ кольцом

Условие:

Для множества $(\mathbb{Z}_{10} = \{0, 1, \dots, 9\})$ с операциями сложения и умножения по модулю 10:

- Проверьте свойства кольца (замкнутость, ассоциативность, дистрибутивность).
- Определите наличие делителей нуля.

Рекомендации:

Аналогично Заданию 3 реализуйте функции по модулю 10.

Задание 9. Проверка, является ли $(\mathbb{Z}_{11})$ полем

Условие:

Поскольку 11 — простое число, проверьте, что $(\mathbb{Z}_{11})$ — поле.

- Найдите обратные элементы по умножению для всех элементов, кроме 0.
- Проверьте основные свойства поля.

Рекомендации:

Используйте расширенный алгоритм Евклида, как в Задании 4.

Задание 10. Исследование подгруппы в группе перестановок (S_4)

Условие:

Рассмотрите группу перестановок (S_4) — все перестановки множества $\{1,2,3,4\}$.

- Найдите подгруппу, состоящую из всех чётных перестановок (группа альтернаций (A_4)).
- Определите порядок подгруппы.

Рекомендации:

- Используйте библиотеку `itertools` для перебора перестановок.
- Определите знак перестановки (чётная или нечётная).
- Соберите все чётные перестановки в подгруппу.

Тест

1) Какая из следующих операций на множестве целых чисел ($\mathbb{Z}$) образует группу?

- A) Сложение ($(+)$)
- B) Умножение ($(\cdot)$)
- C) Вычитание ($(-)$)
- D) Деление ($(/)$)

2) Какой из этих наборов образует поле?

- A) Множество ($\mathbb{Z}_6$) с операциями сложения и умножения по модулю 6
- B) Множество ($\mathbb{Z}_7$) с операциями сложения и умножения по модулю 7
- C) Множество чётных целых чисел с операцией сложения
- D) Множество натуральных чисел с операцией умножения

3) Что такое нейтральный элемент в группе?

- A) Элемент, который является обратным для каждого элемента
- B) Элемент, при операции с которым любой другой элемент остаётся неизменным
- C) Элемент, который отсутствует в группе
- D) Любой элемент группы

4) В множестве ($\mathbb{Z}_{10}$) с операциями по модулю 10 существует ли обратный элемент по умножению для числа 2?

- A) Да
- B) Нет
- C) Обратный элемент есть только для 0
- D) Обратный элемент есть для всех чисел, кроме 0

5) Как называется множество всех чётных целых чисел с операцией сложения?

- A) Кольцо
- B) Поле
- C) Абелева группа
- D) Группа, но не абелева

- 6) Какая из перечисленных структур обязательно содержит делители нуля?
- A) Поле
 - B) Целые числа ($\mathbb{Z}$) с умножением
 - C) Кольцо ($\mathbb{Z}_6$) с операциями по модулю 6
 - D) Множество ненулевых рациональных чисел с умножением
- 7) Какой порядок имеет подгруппа, состоящая из всех 3-циклов в группе перестановок (S_3)?
- A) 2
 - B) 3
 - C) 6
 - D) 1
- 8) Если (p) — простое число, то множество $(\mathbb{Z}_p^*)$ (все ненулевые элементы $(\mathbb{Z}_p)$ с операцией умножения образует:
- A) Кольцо
 - B) Группу
 - C) Поле
 - D) Множество без структуры
- 9) Как называется операция, которая удовлетворяет условию $((a * b) * c = a * (b * c))$ для всех (a, b, c) ?
- A) Коммутативность
 - B) Ассоциативность
 - C) Дистрибутивность
 - D) Нейтральность
- 10) Какой из следующих примеров НЕ является кольцом?
- A) $(\mathbb{Z}, +, \cdot)$ — целые числа с обычными операциями
 - B) $(\mathbb{Z}_4)$ с операциями по модулю 4
 - C) Множество матриц (2×2) с вещественными элементами и обычным сложением и умножением
 - D) Множество натуральных чисел с операцией сложения и умножения